

Day 3 Theory: Servers, APIs, Express and Secrets

Workshop: Web Technology, Hands-on Full-Stack Workshop

Audience: BCA Second Year, Mandsaur University

Time for this day: 2 hours in class, plus about 25 minutes of reading before class

How to use this document

Read it once before class. Do not worry if some parts feel new. Write down your questions and bring them to class.

On Day 2 our Notes Board worked in the browser, but the notes were lost when we refreshed the page. Today we move the notes to a **server**. The browser will **ask** the server for notes, and the server will **answer**. We also add a **password** for adding and deleting notes, and we learn why that password must be kept **secret**.

Remember the habit from Day 2: after every step, do the **Check** line in the lab sheet. Do not move on until the Check is true.

1. Recap of Day 1 and Day 2

- A browser is a **client**. It sends a **request**. A server sends back a **response**.
- **Frontend** is what the user sees. **Backend** is the logic on the server. **Database** stores data.
- On Day 2 we ran our first small server with `node hello.js`. It gave the same answer to every request.
- On Day 2 the notes lived in an **array in the browser's memory**. A refresh made them disappear.

Today's plan:

Browser (frontend)	<---- request and response ---->	Server (backend, Express)
index.html		server.js
app.js (uses fetch)		data.js (the notes)

2. What is an API?

API means **Application Programming Interface**. It is a set of addresses and rules that lets **one program talk to another program**.

Until now, a request to a server gave us a **page** (HTML). A **web API** gives **data** instead. The data is not made for people to read. It is made for programs, such as the JavaScript in our page.

Our Notes Board has an API with three **routes**. A **route** is an address plus a method.

Method	Address	What it does	Password needed?
GET	/api/notes	Gives back all notes	No
POST	/api/notes	Adds a new note	Yes
DELETE	/api/notes/3	Deletes the note with id 3	Yes

Reading is public. Anyone may read the notes. Only a person who knows the password may add or delete.

Why do we start the addresses with `/api` ? It is only a habit. It keeps data addresses apart from page addresses such as `/about.html` .

3. HTTP methods

Every request has a **method**. The method tells the server what the client wants to do. We use three methods today.

Method	Meaning	Example in our app
GET	Read something. It must not change anything	Show all notes
POST	Send new data to be saved	Add a note
DELETE	Remove something	Delete a note

When you type an address in the browser bar and press Enter, the browser sends a **GET** request. That is why we can look at our GET route in the browser. A browser bar cannot send POST or DELETE. For those, we use JavaScript and `fetch` .

There are other methods too. We do not use them today.

4. Status codes

Every response has a **status code**. It is a number that tells the client how the request went.

Code	Name	Meaning	Where we use it
200	OK	It worked	Reading notes
201	Created	It worked, and a new thing was made	A note was added
204	No Content	It worked, and there is nothing to send back	A note was deleted
400	Bad Request	The request is wrong	Title or note is empty or too long
401	Unauthorized	The server does not accept you (here: wrong or missing password)	Wrong password
404	Not Found	There is nothing at that address, or that note does not exist	Deleting a note that is not there

A simple way to remember the groups:

- Codes that start with **2** mean success.
- Codes that start with **4** mean the **client** made a mistake.
- Codes that start with **5** mean the **server** has a problem.

You can see the status code of every request in the browser: press **F12**, open the **Network** tab, and reload the page.

5. JSON

JSON means **JavaScript Object Notation**. It is a way to write data as **text**, so that two programs can send it to each other. Almost every web API uses JSON.

Here is a list of notes in JSON. This is what our `GET /api/notes` route sends:

```
[
  { "id": 2, "title": "Study plan", "text": "Learn HTML on Monday." },
  { "id": 1, "title": "Welcome", "text": "This note came from the server." }
]
```

It looks like a JavaScript array of objects, and that is the idea. There are a few strict rules:

- Names (keys) and text values must use **double quotes**.

- No comments are allowed.
- No comma after the last item.

Two functions turn JavaScript data into JSON text and back:

```
const text = JSON.stringify({ title: "Hi", text: "Hello" }); // object to JSON text
const note = JSON.parse(text);                          // JSON text to object
```

Express and `fetch` do this work for us with `response.json(...)` and `await response.json()`.

6. npm, packages and package.json

Packages

A **package** is code written by other people that we can use in our own project. **npm** is the tool that downloads packages. It came with Node.js on Day 2. npm gets packages from the **npm registry**, which is an online store of packages. This means npm needs the internet.

Today we use one package: **Express**.

package.json

`package.json` is a file that describes our project. It holds the name of the project, the commands we can run, and the list of packages we need. These needed packages are called **dependencies**.

```
{
  "name": "notes-board",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

(This is only a part of the file. The real file has more lines.)

- You make it with `npm init -y`.
- The `-y` means "answer yes to every question".
- The `^` before a version number means "this version, or a newer version of the same major number".

npm install and node_modules

When you run `npm install express`, three things happen:

1. npm downloads Express, and also the other packages that Express needs.
2. It puts them in a folder named `node_modules`.
3. It writes `express` into the `dependencies` of `package.json`, and it writes a file named `package-lock.json`. That file records the exact versions that were installed.

Rules for `node_modules`:

- Never edit anything inside it.
- You do not copy it to other people or upload it. Anyone can make it again by running `npm install` in a folder that has your `package.json`.
- If it is deleted by mistake, run `npm install` again.

Which version do we use?

Our workshop uses **Node.js 24 LTS**. The Express version that we tested the lab with is **5.2.1**. If npm installs a newer version of Express 5 for you, that is fine.

7. Express

Express is a package that makes writing a server much easier. On Day 2 we used only the built-in `http` module and wrote every detail ourselves. Express does the common work for us.

A very small Express server:

```
const express = require("express");

const app = express();

app.get("/hello", function (request, response) {
  response.send("Hello");
});

app.listen(3000);
```

Important words

Word	Meaning
<code>require</code>	Loads a module or a package so that we can use it
<code>app</code>	Our Express server
route	An address plus a method, with a function that answers it
<code>app.get(path, function)</code>	"When a GET request arrives for this path, run this function"
<code>request</code>	Everything the client sent: the path, the headers, the body
<code>response</code>	What we use to answer: <code>response.json(...)</code> , <code>response.status(...)</code>
<code>app.listen(port)</code>	Starts the server and waits for requests

A route with a changing part

In our DELETE route the address is `/api/notes/:id`. The part with the colon, `:id`, changes. For the address `/api/notes/3`, Express gives us `request.params.id`, which is the text `"3"`. We turn it into a number with `Number(...)`.

Our files today

File	Job
<code>server.js</code>	The Express server: the pages and the API routes
<code>data.js</code>	Keeps the notes in memory, with three functions: <code>listNotes</code> , <code>addNote</code> , <code>deleteNote</code>
<code>public/</code>	The folder with the pages that anyone may open: <code>index.html</code> , <code>about.html</code> , <code>css</code> , <code>js</code>
<code>.env</code>	Our private settings (the password). Never uploaded
<code>.env.example</code>	A sample of <code>.env</code> with a fake password. It is safe to share
<code>.gitignore</code>	A list of files that Git must never upload

Modules: sharing code between files

In Node.js each file is a **module**. A file can **export** things, and another file can **require** them.

```
// data.js (at the end of the file)
module.exports = { listNotes, addNote, deleteNote };

// server.js
const { listNotes, addNote, deleteNote } = require("./data");
```

The `./` means "in the same folder as this file".

Why a `public` folder?

Every file in `public` is sent to **any visitor** who asks for it. So we put only the pages there. We never put `server.js`, `data.js` or `.env` in `public`. If we did, a visitor could read our code and our password.

8. Middleware

Middleware is a function that runs **between** the request and the final answer. It can look at the request, change it, stop it, or let it go on. To let it go on, it calls `next()`.

Think of a security guard at a gate. The guard looks at each visitor. A good visitor is let in. A bad visitor is stopped at the gate.

```
request -> express.json -> express.static -> requirePassword -> route function
-> response
```

We use three middleware functions:

Middleware	What it does
<code>express.json()</code>	Reads JSON sent by the browser and puts it in <code>request.body</code>
<code>express.static(folder)</code>	Sends the files of a folder (our <code>public</code> folder) when the address matches a file
<code>requirePassword</code> (ours)	Checks the password. If it is wrong, it answers 401 and stops. If it is right, it calls <code>next()</code>

Our own middleware:

```
function requirePassword(request, response, next) {
  if (request.get("x-notes-password") !== PASSWORD) {
    response.status(401).json({ error: "Wrong password." });
    return;
  }
  next();
}
```

We put it in front of only the two routes that change data:

```
app.post("/api/notes", requirePassword, async function (request, response) { ... });
app.delete("/api/notes/:id", requirePassword, async function (request, response) { ...
});
```

The `GET` route has no `requirePassword`, so reading stays public.

Order matters. Express reads our code from top to bottom. `express.json()` must be placed **before** the routes that use `request.body`.

9. async, await and fetch

Why waiting?

Some jobs take time: asking a server for data, or (from Day 4) reading a database. A program must not freeze while it waits. JavaScript solves this with **promises**.

A **promise** is an object that means "the answer is not ready yet, but it will come". We can **wait** for it with `await`.

```
async function loadNotes() {
  const response = await fetch("/api/notes"); // wait for the server
  const notes = await response.json();       // wait for the JSON to be read
  console.log(notes);
}
```

- A function marked `async` always gives back a promise.
- `await` can be used only inside an `async` function. It waits for the promise and gives us the result.
- `try` and `catch` handle problems, for example when the server is not running.

Why are the functions in data.js async?

Today `data.js` keeps the notes in an array, so nothing really needs to wait. We still write `async` now, because on Day 4 these same three functions will talk to a database, and a database takes time. Because the names and the way of calling stay the same, `server.js` will need almost no change.

fetch

`fetch` is the browser function that sends a request to a server. Reading notes (GET is the default method):

```
const response = await fetch("/api/notes");
const notes = await response.json();
```

Sending a new note (POST):

```
const response = await fetch("/api/notes", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "x-notes-password": passwordInput.value
  },
  body: JSON.stringify({ title: title, text: text })
});
```

- `method` is the HTTP method.
- `headers` are extra information sent with the request. `Content-Type` tells the server that the body is JSON. `x-notes-password` is our own header that carries the password.
- `body` is the data, as JSON text.
- `response.ok` is `true` when the status code is in the 200 range.
- `response.status` is the status code number.

An address that starts with `/`, such as `/api/notes`, means "on the same server that sent this page".

10. Secrets, environment variables and .env

What is a secret?

A **secret** is a value that only the right people may know: a password, an API key, a database address with a password inside. If a secret becomes public, other people can use it.

Never put a secret in your code

If we write `const PASSWORD = "abc123";` in `server.js`, then:

- Everyone who gets a copy of the code gets the password.
- When we upload the project to GitHub (Day 4), the password goes to the internet. If the repository is public, **anyone** can read it.
- Even if we delete the password later, the older versions of the file are still kept in Git's history.

So the password must live **outside** the code.

Environment variables

An **environment variable** is a named setting that lives outside your program, in the environment where the program runs. Node.js reads them with `process.env`:

```
const PASSWORD = process.env.NOTES_PASSWORD;
```

Your code only holds the **name** `NOTES_PASSWORD`. The **value** comes from outside. A hosting service has a place to type such settings. We use it on Day 5.

The .env file

On our own computer we keep the settings in a small text file named `.env` (the name starts with a dot):

```
NOTES_PASSWORD=choose-your-own-password
PORT=3000
```

- One setting per line, written as `NAME=value`.
- No spaces are needed around the `=`.
- Node.js 24 can read this file with one line: `process.loadEnvFile()`. We observed this working with our code on Node.js 24.21.0.
- A server reads the `.env` file only when it **starts**. If you change `.env`, restart the server.

.env.example

We cannot share `.env`, but other people need to know which settings exist. So we also write `.env.example`. It has the same names but a **fake** value. It is safe to share. A new person copies it to `.env` and writes a real value.

.gitignore

Git is a tool that records every version of your project. We start using it on Day 4. A file named `.gitignore` lists the files and folders that Git must **ignore**. Ours lists:

```
node_modules/
.env
```

`node_modules/` can be made again with `npm install`. `.env` is private. We write `.gitignore` today, **before** we ever use Git, so we cannot upload the password by mistake.

Rule: A password goes in `.env`. `.env` goes in `.gitignore`. Never upload it.

Stop early when the password is missing

Our `server.js` checks the password at the start. If `NOTES_PASSWORD` is empty, the server prints a clear message and stops. This is better than running with no password.

11. Why we check the password on the server

Our page has a password box. A common beginner idea is: "The browser JavaScript compares the password, and if it is right, it allows the note." **This is not safe.**

- Everything in the browser belongs to the user. Anyone can press **F12**, read our JavaScript, and **change** it.
- If the real password is inside the browser JavaScript, everyone who opens the page gets it.
- Anyone can also send a request to our API **without our page at all**, for example from the browser Console. Our page is only one way to use the API.

So the **server** must make the decision. In our app:

1. The user types the password in the page.
2. `fetch` sends it in the `x-notes-password` header.
3. The server compares it with the real password, which only the server knows.
4. If it is wrong, the server answers **401** and does nothing.

The same rule applies to the checks on the title and the note. The browser stops an empty title to be friendly to the user. The server checks again, because the server cannot trust the browser. Our server also refuses a title longer than 60 letters and a note longer than 300 letters.

This is a simple lock on purpose. One shared password for everyone is enough to learn the idea. Real applications use separate accounts and use a secure connection (HTTPS). We do not cover that in this workshop.

12. Safety again: textContent

On Day 2 we learned that user text must be put into the page with `textContent`. Today the notes come from a server, but the rule is the same. A note could contain `<b>bold</b>` or even harmful code. If we put it in the page as HTML, the browser may run it. This is called **XSS**.

```
heading.textContent = note.title;    // SAFE
// heading.innerHTML = note.title;    // UNSAFE with user input
```

In our lab you add a note with the text `<b>bold</b>`, and the page shows it as plain letters. We tested this on the finished code.

Remember: **user input is never trusted**, and data that comes from a server is also user input if users put it there.

13. What is still missing?

Run the lab and you will see two things:

1. You add a note, refresh the page, and the note is **still there**. This is the improvement. The notes now live on the server.
2. You stop the server, start it again, and the added notes are **gone**.

Why? `data.js` keeps the notes in an **array in the memory of the server program**. When the program stops, its memory is cleared. Memory is fast, but it is not permanent.

To keep notes after the server stops, we must write them on the **disk**, in a **database**. That is the plan for **Day 4**. Because `data.js` has only three functions, Day 4 changes only the inside of those three functions.

14. Quick revision

1. An **API** is a set of routes that lets programs talk to a server and exchange **data**.
 2. **GET** reads, **POST** adds, **DELETE** removes.
 3. Status codes: **200** OK, **201** Created, **204** No Content, **400** Bad Request, **401** Unauthorized, **404** Not Found.
 4. **JSON** is text for data. `JSON.stringify` and `response.json()` convert it.
 5. **npm** downloads packages into **node_modules**. **package.json** lists them. Never upload `node_modules`.
 6. **Express** makes servers easy: `app.get`, `app.post`, `app.delete`, `app.listen`.
 7. **Middleware** runs between the request and the answer. Our `requirePassword` is one.
 8. **async/await** waits for slow jobs. **fetch** sends a request from the browser.
 9. A **secret** goes in `.env`, and `.env` goes in `.gitignore`.
 10. The password is checked on the **server**, never in the browser JavaScript.
 11. Use `textContent` for user text.
 12. Notes in the server's memory are lost when the server stops. Day 4 fixes this with a database.
-

15. Check yourself

Try to answer these without looking above. Write your answers in your notebook.

1. What does API stand for? Explain it in one sentence.
 2. Which HTTP method do we use to add a note? Which one to delete a note?
 3. What does the status code **401** mean in our app? What does **404** mean?
 4. Why do we need `express.json()` before the POST route?
 5. What is the job of `requirePassword`? What does `next()` do?
 6. What is inside the `node_modules` folder? Why do we not upload it?
 7. Why is the password in `.env` and not in `server.js`?
 8. Why does `.gitignore` list `.env`?
 9. Why must the password be checked on the server and not in the browser JavaScript?
 10. After you restart the server, the notes you added are gone. Why? What will fix it?
-

16. Homework for Day 4

1. Check that your **GitHub account** exists (you were asked to create it before Day 3). Write down your **username**. We use GitHub on Day 4.
2. Run your Notes Board at home or in the lab once more. It must start with `node server.js` and let you add a note with your password.

3. Answer the **Check yourself** questions in your notebook.
 4. Try the **extra challenges** at the end of the Day 3 Lab Sheet.
 5. Write down at least two questions for Day 4.
-

Glossary

Word	Meaning
API	Application Programming Interface: addresses and rules that let one program talk to another program
async	A word that marks a function which gives back a promise
await	Waits for a promise to finish and gives its result
Body	The data sent with a request or a response
Dependency	A package that your project needs
DELETE	The HTTP method for removing something
Environment variable	A named setting that lives outside your code
.env	A private text file with settings such as the password
.env.example	A sample of <code>.env</code> with fake values. Safe to share
Express	A package that makes writing a Node.js server easy
fetch	The browser function that sends a request to a server
.gitignore	A file that lists what Git must not upload
GET	The HTTP method for reading something
Header	Extra information sent with a request or a response
HTTP method	The action of a request, such as GET, POST or DELETE
JSON	JavaScript Object Notation: data written as text
Middleware	A function that runs between the request and the answer
Module	A JavaScript file that can share code with other files
node_modules	The folder where npm puts downloaded packages
npm	The tool that downloads packages
Package	Ready-made code from other people
package.json	The file that describes a project and its dependencies
POST	The HTTP method for sending new data
Promise	An object that means "the answer will come later"

Word	Meaning
Route	An address plus a method, with a function that answers it
Secret	A value that only the right people may know
Status code	A number in the response that tells how the request went
XSS	An attack that runs harmful code by using unsafe user input